

E-Commerce CLI - Bug Report & Fixes

This document tracks the recently discovered bugs and user experience (UX) friction points, outlining their root causes and the specific code changes implemented to resolve them.

Bug 1: `NumberFormatException` Crash on CSV Load

Status: Fixed

Severity: Critical (System Failure)

Description:

When the `products.csv` dataset was upgraded to include a 4th column for "Category" (`id,name,category,price`), the application immediately threw a `NumberFormatException` and skipped all 1,000 valid products, leaving the catalog completely empty.

Root Cause:

`FileHandler.loadProducts()` was hardcoded to read the price from the 3rd column (`data[2]`). With the new schema, the 3rd column contained String data (e.g., `"Electronics"`), causing `Double.parseDouble("Electronics")` to crash.

Resolution:

Updated the parsing logic in `FileHandler.java` to dynamically check the array length. If `data.length >= 4`, it safely parses the Category from `data[2]` and the Price from `data[3]`. It remains backwards compatible with the older 3-column CSV format.

Bug 2: Tedious Product ID Entry (UX Issue)

Status: Fixed

Severity: Medium (Friction)

Description:

Users were required to perfectly type out the full Product ID (e.g., `P001`, `P047`) exactly as formatted in the CSV. Simply entering `1` or `47` resulted in an "Invalid Product ID" error, which slowed down the checkout process.

Root Cause:

The search logic utilized strict string matching (`p.getId().equalsIgnoreCase(input)`), completely failing on raw numeric inputs.

Resolution:

Implemented a smart regex-based auto-padding algorithm. If the user input matches pure digits (`input.matches("\\d+")`), the system wraps the integer using `String.format("P%03d", Integer.parseInt(input))` . This allows users to seamlessly type `1` and have it automatically resolved to `P001` .

Bug 3: Error-Prone Coupon Code Input (UX Issue)

Status: Fixed

Severity: Low (Friction)

Description:

During the final checkout phase, users were forced to manually type out complex coupon strings (e.g., `MYFIRST60` , `WINTER25`). This was tedious and highly prone to typing errors, causing valid discounts to be rejected.

Root Cause:

The coupon loop only verified string equality and provided no alternative selection method for the user.

Resolution:

Overhauled the coupon display loop in `main.java` to assign and print a 1-based index (Serial Number) next to each valid coupon. Added intelligent parsing logic so if a user types a number (e.g., `1`), the system automatically grabs that coupon from `coupons.get(serial - 1)` . The original string fallback was retained so users can still manually type the code if preferred.

Legacy Bugs & Design Adjustments (From Previous Implementations)

Bug 4: Cluttered Checkout with Tax & Inventory Logic (Design Issue)

Status: Resolved (Rolled Back)

Severity: Medium (UX Bloat)

Description:

The application initially featured complex inventory tracking, tax calculations, and multi-tier coupon systems. This overly complicated the core user flow, making the checkout process feel bloated rather than a clean, interactive CLI.

Root Cause:

Over-engineering the baseline requirements of the assignment.

Resolution:

Per user request, the complex inventory and tax systems were explicitly rolled back. The codebase was simplified to prioritize a clean, direct, and user-friendly adding and checkout experience.

Bug 5: Missing Persistent Receipt Storage & Branding

Status: Fixed

Severity: Low (Feature Missing)

Description:

Receipts were only being printed directly to the console and lacked a personal touch, making it difficult to save or verify past checkout attempts.

Root Cause:

No file I/O output stream was hooked into the `Cart.generateReceipt()` completion phase.

Resolution:

Added `FileHandler.writeOutput()` to automatically export every successful checkout directly into `output/sample_outputs.txt`. Additionally, integrated a standardized, personalized `"THANK YOU :)"` footer to the receipt generator to improve the polished feel of the final output.